Week 6 — Web Security & Vulnerability Testing

Vulnerability Assessment & Penetration Testing (VAPT Basics)

Student	SAMYUKTHA S.
Internship ID	DG/SEPTEMBER/CYBER/102
Programme	DG Interns Hub – Cybersecurity Internship
Target	`http://127.0.0.1/dvwa/`
Environment	Local DVWA lab
Report date	13 October 2026

Assessment disclaimer

All practical testing documented in this report was conducted against an intentionally vulnerable application in a controlled local lab. The results are educational observations and must not be interpreted as evidence of vulnerabilities in a production system. Automated scanner alerts that have not been manually validated are not confirmed vulnerabilities.

Table of Contents

Section	Title
1	Objective
2	Scope and Authorization
3	Lab Environment and Tools
4	Methodology
5	Practical Testing and Results
5.1	SQL Injection
5.2	Reflected Cross-Site Scripting (XSS)
5.3	Stored Cross-Site Scripting (XSS)
6	Low / Medium / High Security-Level Comparison
7	Burp Suite Request and Response Analysis
8	OWASP ZAP Scan
9	Findings Register
10	Remediation Recommendations
11	Evidence Index
12	Learning Outcomes
13	Conclusion
14	References

1. Objective

The objective of this practical exercise was to build foundational web application security testing skills in a controlled DVWA lab. The work focused on observing SQL Injection and Cross-Site Scripting behavior, inspecting HTTP traffic with Burp Suite Community Edition, and reviewing a partial OWASP ZAP scan.

- Demonstrate how selected inputs behave in intentionally vulnerable DVWA modules.
- Compare observed behavior across DVWA Low, Medium, and High security settings without overstating unverified outcomes.
- Capture and replay a SQL Injection HTTP request using Burp Suite Repeater.
- Record scanner output, evidence locations, limitations, and practical remediation recommendations.

2. Scope and Authorization

The assessment scope was restricted to the local Damn Vulnerable Web Application (DVWA) instance at the loopback address shown below. The archive and project notes describe this as an educational, authorized local lab.

Scope item	Details
Target application	Damn Vulnerable Web Application (DVWA)
Target URL	http://127.0.0.1/dvwa/
Testing boundary	Local application only; no production or third-party target included
Assessment purpose	Educational practice, evidence collection, and remediation learning

> Testing disclaimer: Tools and payloads were used only against the local training application. The results do not constitute a production security audit.

3. Lab Environment and Tools

Component	Use in this exercise
Kali Linux local lab	Testing environment described in the project documentation
DVWA	Intentionally vulnerable web application; security level selected per exercise
Burp Suite Community Edition	HTTP history, request inspection, and Repeater replay
OWASP ZAP 2.17.0	Automated scan and HTML report export; scan was stopped before completion
Screenshots and project archive	Stored evidence for observed UI behavior and tool output

4. Methodology

1. Confirm access to the local DVWA instance and review the configured security level.
2. Use normal inputs as a baseline, then submit the recorded test payloads in the relevant DVWA modules.
3. Observe browser output and application responses; capture screenshots for the documented tests.
4. Capture and replay a SQL Injection HTTP request in Burp Suite Repeater.
5. Run OWASP ZAP against the local target, export the report, and preserve the scan-completion limitation.
6. Separate demonstrated Low-level behavior from Medium/High observations and unvalidated automated alerts.
7. Recommend remediation and retesting, without claiming fixes that were not implemented and verified.

Lab Setup and Security-Level Evidence

The supplied project archive contains four genuine DVWA setup screenshots. They are included here as contextual evidence of the local lab setup and the available DVWA security-level pages; they do not independently validate a vulnerability test result.

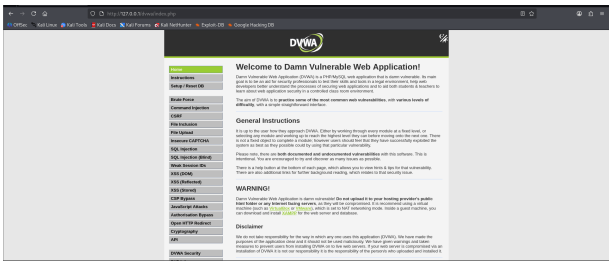


Figure A. DVWA setup-success screen.

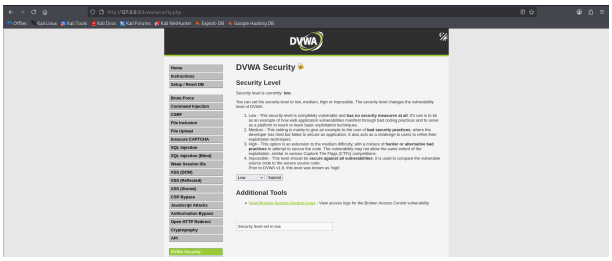


Figure B. DVWA Low security-level screen.

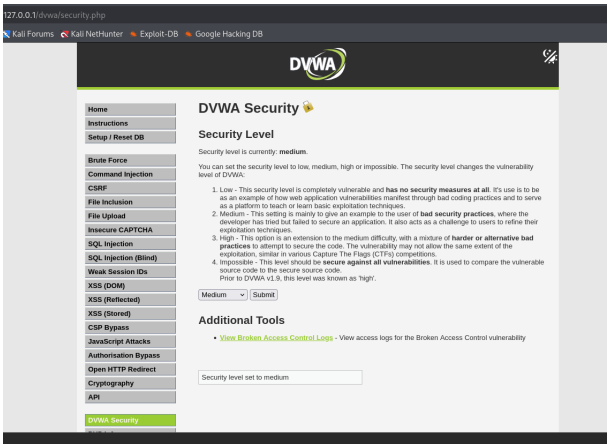


Figure C. DVWA Medium security-level screen.

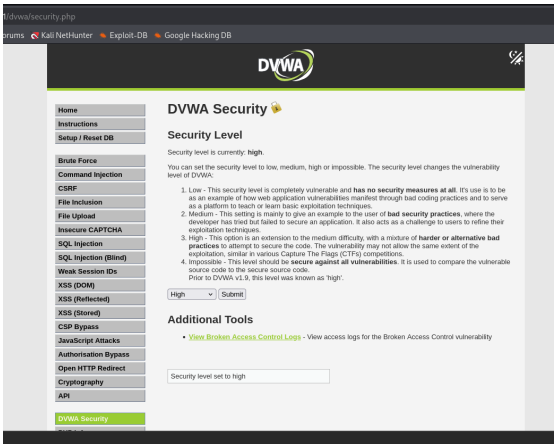


Figure D. DVWA High security-level screen.

These screenshots establish the presence of setup/security-level evidence in the project archive. The Medium/High SQL Injection interpretation remains pending validation as described in the report.

5. Practical Testing and Results

5.1 SQL Injection — Low security

The SQL Injection module was tested with a normal input and a Boolean-style test payload while DVWA was set to Low security. The project notes record that normal input `1` returned an individual record, while `1' OR '1'='1' #` returned multiple records. This is a demonstrated behavior in the intentionally vulnerable local lab.

Observed result

- Baseline: input `1` returned an individual record.
- Test: payload `1' OR '1'='1' #` returned multiple records.
- A corresponding HTTP request was captured and replayed in Burp Suite Repeater.
- The result demonstrates SQL Injection behavior in this lab configuration only.

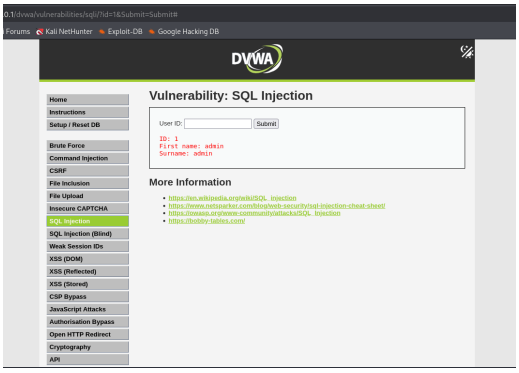


Figure 1. DVWA SQL Injection module — normal-input baseline.

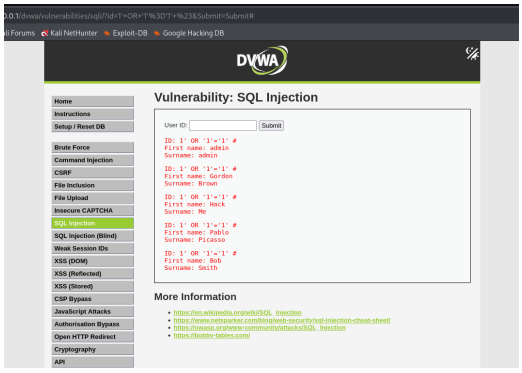


Figure 2. DVWA SQL Injection module — payload test output.

Potential impact

In a real application, unsafe query construction may allow an attacker to retrieve or modify database information or bypass data restrictions. Actual impact depends on query structure, database privileges, and application design.

Validation statement

> Confirmed only as a demonstration in the local DVWA Low-security exercise. No production vulnerability or successful remediation is claimed.

5.1 SQL Injection — Medium and High observations

The Medium folder contains screenshots for IDs 1–5. The user reported that five IDs appeared after using the popup input. The exact input and resulting output cannot be conclusively reconciled from the supplied description alone; therefore this is recorded as an observation requiring validation rather than a confirmed successful or blocked injection.

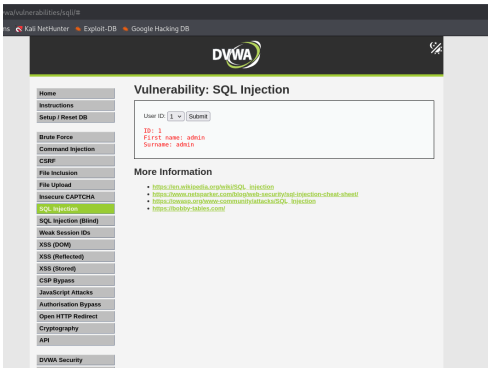


Figure 3. Medium security — saved output for ID 1.

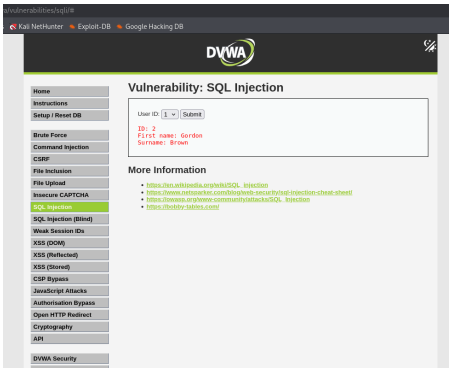


Figure 4. Medium security — saved output for ID 2.

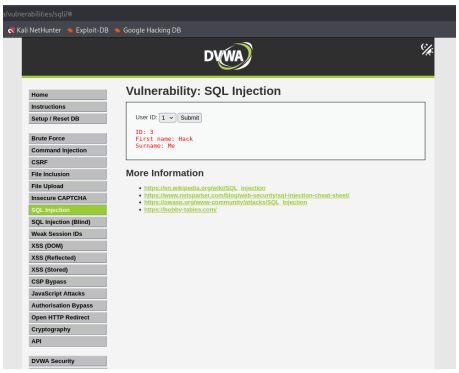


Figure 5. Medium security — saved output for ID 3.

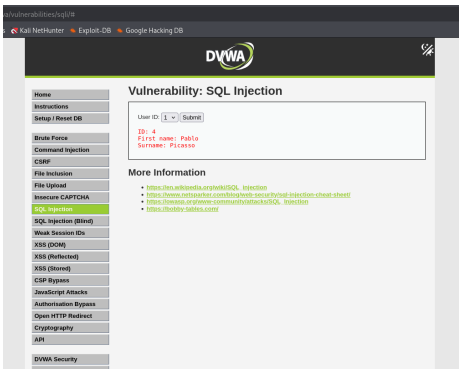


Figure 6. Medium security — saved output for ID 4.

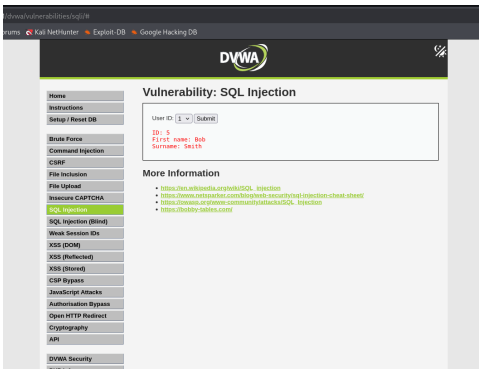


Figure 7. Medium security — saved output for ID 5.

SQL Injection — High security observation

Normal-input and payload-test screenshots were saved in the High folder. The user reported that five IDs appeared, but the exact request/response behavior has not been independently verified. No claim is made that injection definitively succeeded or was blocked.

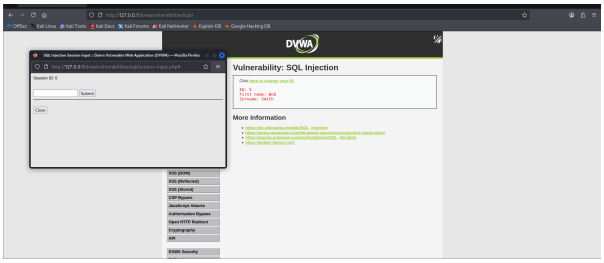


Figure 8. High security — normal-input test screenshot.

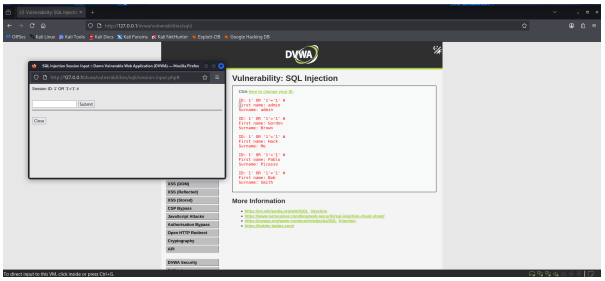


Figure 9. High security — payload-test screenshot.

Medium/High SQL Injection status: observation recorded; exact input/output interpretation requires validation. A screenshot alone should not be treated as proof of exploitability or effective blocking.

5.2 Reflected Cross-Site Scripting (XSS)

Reflected XSS may occur when untrusted input is returned in a response and interpreted by the browser as executable content. The test payload used was `<script>alert('Reflected XSS')</script>`.

Low security — demonstrated

A browser alert appeared during the Low-security test. The screenshot below records the alert displayed in the local DVWA browser.

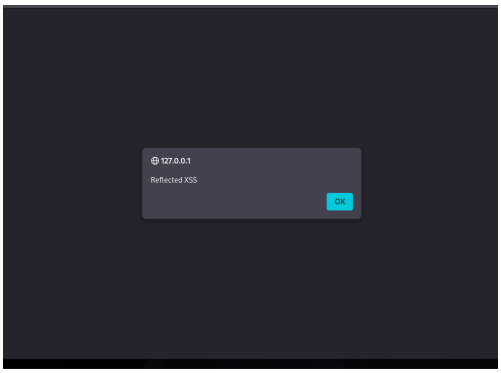


Figure 10. Reflected XSS — browser alert observed at Low security.

Medium security — filtered/displayed as text

The supplied project description states that the payload was filtered or displayed as text and no JavaScript alert was observed. The screenshots document the normal input, special-character behavior, and payload test.

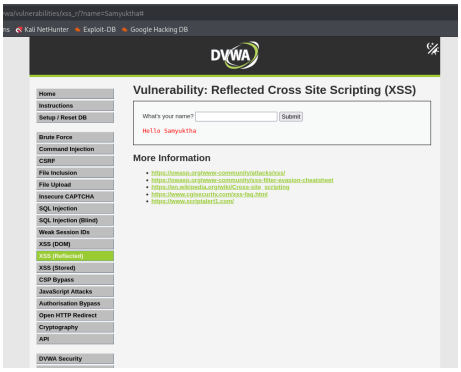


Figure 11. Reflected XSS Medium — normal input.

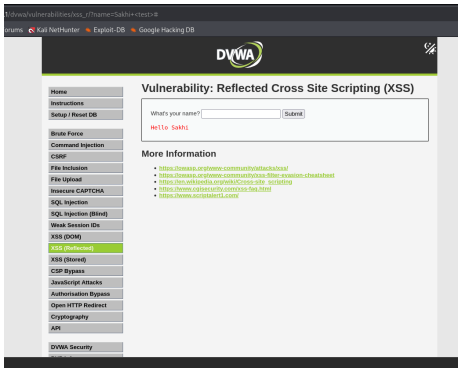


Figure 12. Reflected XSS Medium — special-character test.

Reflected XSS — Medium evidence and High observation

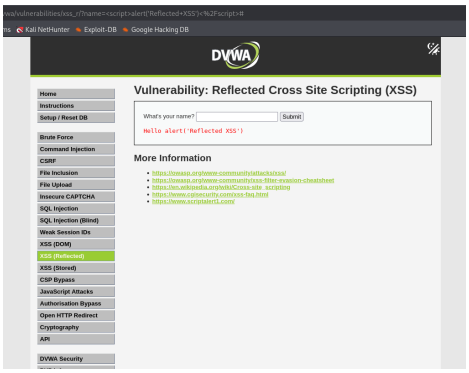


Figure 13. Reflected XSS Medium — payload-test output.

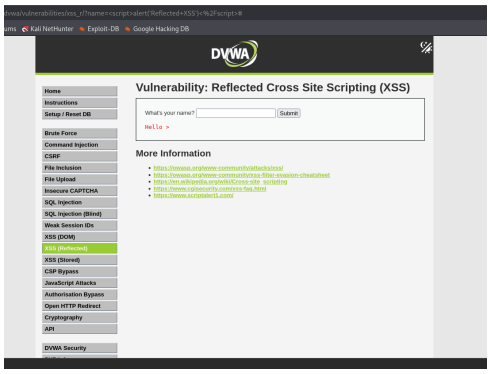


Figure 14. Reflected XSS High — payload-test output.

For the High-security test, the observed output was `Hello >` and no alert was reported. This indicates altered or filtered output for this particular payload. It does not prove that all XSS payloads or contexts are prevented.

Potential impact

In a vulnerable real application, XSS can execute attacker-controlled script in a user's browser under the affected site's origin. Actual impact depends on the vulnerable context, user privileges, browser controls, and application behavior.

> Only the Low-security Reflected XSS alert is documented as a demonstrated execution. Medium and High observations are limited to the specific tests described.

5.3 Stored Cross-Site Scripting (XSS)

Stored XSS may occur when user-supplied content is saved and later rendered without appropriate context-aware output encoding or sanitization. The test payload used was `<script>alert('Stored XSS')</script>` through the DVWA guestbook module.

Low security — demonstrated

The stored guestbook payload triggered a browser alert when the stored content was rendered. This demonstrates script execution in the local DVWA Low-security exercise.

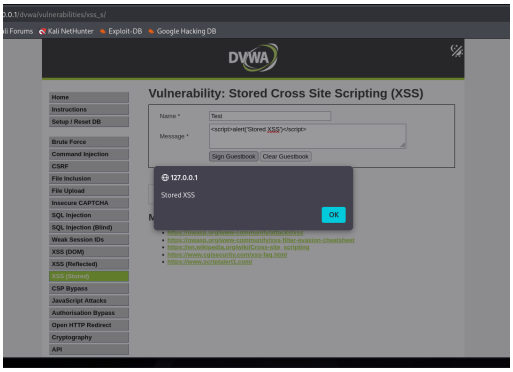


Figure 15. Stored XSS — browser alert observed when stored guestbook content rendered.

Medium security — text only

The provided description states that the payload appeared as text and no JavaScript execution was observed. The screenshots below preserve the normal-input and payload-test evidence.

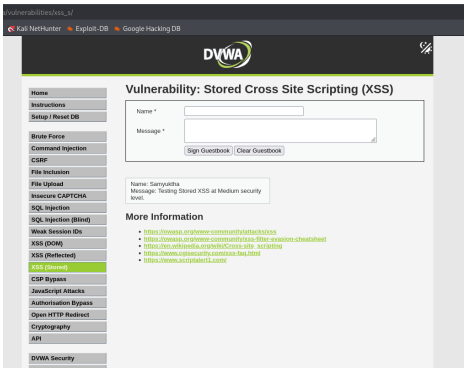


Figure 16. Stored XSS Medium — normal-input baseline.

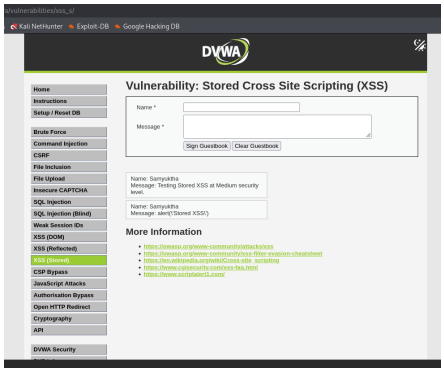


Figure 17. Stored XSS Medium — payload displayed as text; no execution observed.

Stored XSS — High observation

The High-security screenshot shows the payload test. The provided observation is that the payload appeared only as text and no JavaScript execution was observed. This result applies to the tested payload and context only.

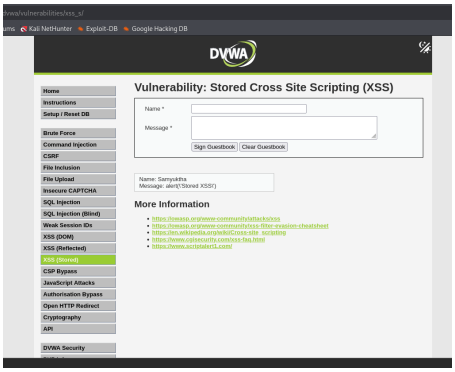


Figure 18. Stored XSS High — payload-test output.

Potential impact

Stored XSS may affect users who view the stored content. Depending on the application, the impact may include unauthorized actions in a user's browser context or access to page data available to that user.

> Only the Low-security Stored XSS alert is documented as a demonstrated execution. Medium/High results are observations for the supplied payload and do not establish universal protection.

6. Low / Medium / High Security-Level Comparison

The table summarizes only the recorded observations. The comparison is not a complete security evaluation of DVWA's security levels.

Test	Low	Medium	High
SQL Injection	Normal input returned one record; payload returned multiple records. Demonstrated in lab.	Screenshots for IDs 1–5; user reported five IDs appeared. Exact input/output interpretation requires validation.	Normal and payload screenshots saved; user reported five IDs. Exact behavior requires validation.
Reflected XSS	Browser alert appeared for the test payload.	Payload described as filtered/displayed as text; no alert observed.	Output observed as `Hello >`; no alert reported. Only this payload/context assessed.
Stored XSS	Stored guestbook payload triggered a browser alert when rendered.	Payload appeared as text; no JavaScript execution observed.	Payload appeared as text; no JavaScript execution observed.

Interpretation limits

- DVWA security levels are training configurations; their behavior is not a substitute for testing a real application.
- The supplied SQL Injection Medium/High screenshots and reported five-ID observation do not independently establish success or blocking.
- No alert for a single XSS payload is not proof that every payload, encoding, or rendering context is safe.
- No severity score is assigned to the security-level comparison because production context and validated exploitability are not established.

7. Burp Suite Request and Response Analysis

Burp Suite Community Edition was used to inspect local DVWA traffic, capture a SQL Injection request, and replay it in Repeater. The project notes report that the response returned HTTP 200 and multiple records.

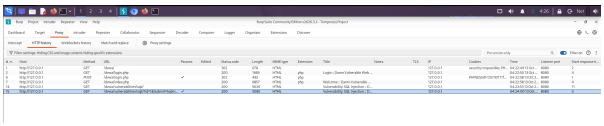


Figure 19. Burp Suite HTTP history showing captured local traffic.

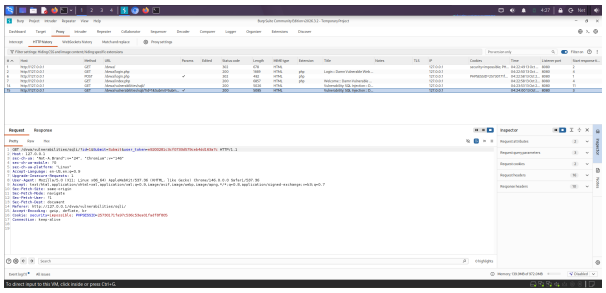


Figure 20. SQL Injection request captured for inspection.

Burp Suite — Repeater evidence

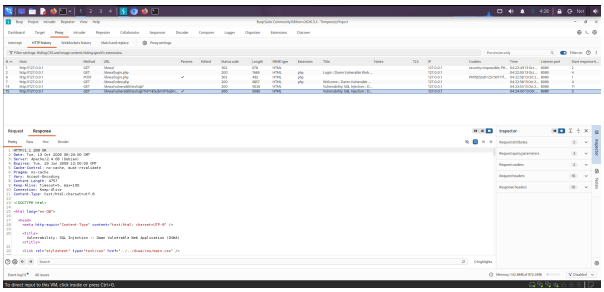


Figure 21. Response associated with the SQL Injection request.

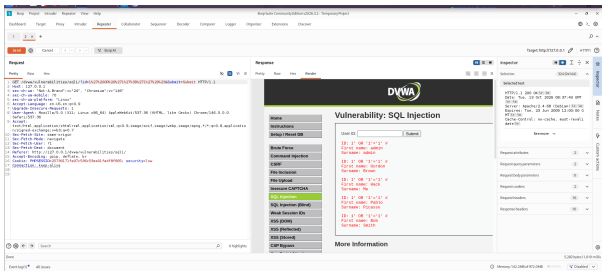


Figure 22. Repeater request/response view used to replay the local request.

Analysis

- HTTP history provides a record of requests made by the browser through the configured proxy.
- Repeater supports controlled replay of a request to compare application responses to changed inputs.
- The evidence supports that request inspection and replay were performed; interpretation remains bounded to the local DVWA lab.
- The recommended follow-up is to repeat the same baseline and test cases after remediation, then compare status, response content, and application behavior.

8. OWASP ZAP Scan

OWASP ZAP version 2.17.0 was used against the local DVWA target at 127.0.0.1. The scan was stopped before completion and must be treated as a partial scan.

Reported alert category	Count
High	0
Medium	5
Low	5
Informational	3
Total	13

The saved summary also reported 5 high-confidence alerts and 8 medium-confidence alerts. These confidence labels describe the scanner's confidence classification, not the severity category.

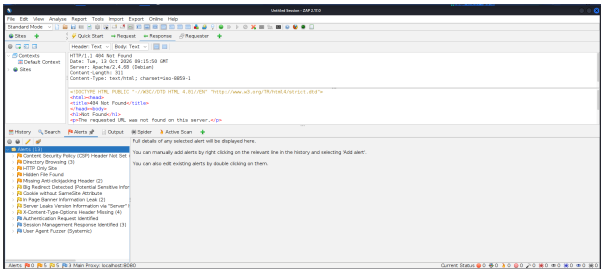


Figure 23. OWASP ZAP alert summary captured during the partial scan.

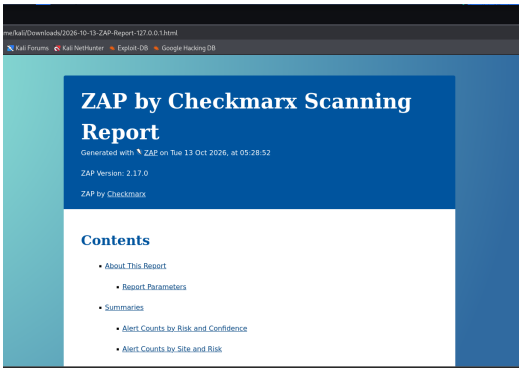


Figure 24. Generated ZAP HTML report evidence.

Report file: `03-Reports/2026-10-13-ZAP-Report-127.0.0.1.html`

Limitations and next steps

- The scan was manually stopped before completion; it is not a full coverage assessment.
- The 13 alerts are not 13 confirmed exploitable vulnerabilities.
- Individual alerts must be reviewed for affected URL, evidence, confidence, and reproducibility.
- No individual alert details are asserted here because the available summary does not establish which alerts were manually confirmed.
- The absence of High alerts in this partial scan does not prove the application is secure.

9. Findings Register

The register separates demonstrated local-lab behavior from observations that need further validation. These labels are evidence-status descriptions, not formal production severity ratings.

ID	Finding / observation	Evidence status	Priority / next step
WEB-01	SQL Injection — DVWA Low	Demonstrated: baseline 1 returned one record; payload returned multiple records; Burp request replay evidence saved.	Validate Medium/High observations separately; remediate query construction.
WEB-02	Reflected XSS — DVWA Low	Demonstrated: test payload triggered browser alert.	Retest output encoding in the affected rendering context.
WEB-03	Stored XSS — DVWA Low	Demonstrated: stored guestbook payload triggered alert when rendered.	Retest submission and every relevant display location.
WEB-04	OWASP ZAP alerts	Partial scan; 13 alerts reported; individual alert validation pending.	Triage each alert; distinguish confirmed issues, false positives, and informational items.
WEB-05	Broken Authentication	Theory reviewed; dedicated practical assessment not confirmed.	Do not claim a finding; test only if an authorized plan is conducted.
WEB-06	Security Misconfiguration	Potential observations; no individual alert confirmed from the summary alone.	Review configuration and validate each relevant alert.

The Low-security SQL Injection and XSS demonstrations are findings within the intentionally vulnerable training application. They are not claims about any production system. No remediation is marked complete.

10. Remediation Recommendations

Area	Recommendation	Retesting approach
SQL Injection	Use parameterized queries/prepared statements; avoid concatenating untrusted input into SQL; apply least-privilege database permissions; avoid exposing database errors.	Repeat the same baseline input and authorized test cases. Verify inputs are treated as data and results are appropriately constrained.
Reflected XSS	Apply context-aware output encoding; validate inputs by expected format; use safe templating and DOM APIs; avoid unsafe HTML insertion.	Retest the same payload in the affected output context and verify it is rendered as inert text where appropriate.
Stored XSS	Encode stored data when rendered. If rich text is intentionally supported, use a maintained allow-list HTML sanitizer. Prefer safe rendering APIs.	Retest submission and all relevant pages that display stored content; verify stored content remains inert.
Defense in depth	Use Content Security Policy as an additional layer, not as a replacement for output encoding or safe DOM practices.	Verify policy behavior and ensure required application functionality continues to work.
OWASP ZAP	Manually validate alerts; review affected URL, evidence, confidence, and suggested fix; document false positives and informational observations.	Reproduce relevant alerts in the authorized lab and run a complete follow-up scan when practical.
Security configuration	Review server defaults, unnecessary functionality, security headers, permissions, error handling, and exposed information.	Verify the actual configuration and retest the affected behavior after changes.

These are recommendations, not evidence of completed fixes. Mark a finding as remediated only after a change has been implemented and supported by retesting evidence.

11. Evidence Index

The following image files were present in the supplied ZIP archive and are embedded in the relevant report sections. Paths are relative to `Week-06-Web-Security-Vulnerability-Testing/`.

Evidence group	Files included	Report figure(s)
00-Lab-Setup	DVWA-Setup-Success.png; DVWA-Low-Security-Level.png; DVWA-Medium-Security-Level.png; DVWA-High-Security-Level.png	Lab setup screenshots are in the archive; selected security-level context is reflected in the comparison. The setup images are listed here rather than presented as decorative visuals.
01-SQL-Injection/Low	SQLi-01-Normal-Input.png; SQLi-02-Injection-Test.png	Figures 1–2
01-SQL-Injection/Medium	ID=1.png; ID=2.png; ID=3.png; ID=4.png; ID=5.png	Figures 3–7
01-SQL-Injection/High	SQLi-Normal-Input-Test.png; SQLi-High-Payload-Test.png	Figures 8–9
02-Reflected-XSS/Low	Reflected-XSS-01-Alert-Executed.png	Figure 10
02-Reflected-XSS/Medium	XSS-Reflected-Medium-Normal-Input.png; XSS-Reflected-Medium-Special-Characters.png; XSS-Reflected-Medium-Payload-Test.png	Figures 11–13
02-Reflected-XSS/High	XSS-Reflected-High-Payload-Test.png	Figure 14
03-Stored-XSS/Low	Stored-XSS-01-Alert-Executed.png	Figure 15
03-Stored-XSS/Medium	XSS-Stored-Medium-Normal-Input.png; XSS-Stored-Medium-Payload-Test.png	Figures 16–17
03-Stored-XSS/High	XSS-Stored-High-Payload-Test.png	Figure 18
04-Burp-Suite	Burp-01-HTTP-History.png; Burp-02-SQLi-Request.png; Burp-03-SQLi-Response.png; Burp-04-SQLi-Repeater-Request-Response.png	Figures 19–22
05-OWASP-ZAP	ZAP-01-Alert-Summary.png; ZAP-02-Report-Generated.png	Figures 23–24

12. Learning Outcomes

- Practised manual input testing against SQL Injection and XSS modules in a deliberately vulnerable application.
- Observed the difference between a baseline request and a test payload in the Low-security SQL Injection exercise.
- Observed browser-side script execution in the Low-security Reflected and Stored XSS exercises.
- Captured and replayed an HTTP request using Burp Suite, reinforcing the role of request/response inspection in web testing.
- Generated and exported an OWASP ZAP report, while recognizing that a partial scan and unvalidated alerts cannot establish a complete security posture.
- Learned to distinguish demonstrated behavior, reported observations, theory-only topics, and unvalidated scanner output.
- Practised evidence-based reporting and remediation planning without claiming unverified findings or completed fixes.

13. Conclusion

The Week 6 exercises provided hands-on exposure to foundational web application testing in a controlled local DVWA lab. SQL Injection, Reflected XSS, and Stored XSS behavior were demonstrated at the Low security setting, and Burp Suite was used to inspect and replay a SQL Injection request. OWASP ZAP produced a report from a partial scan.

The Medium and High security-level observations are recorded with appropriate uncertainty, particularly for SQL Injection where the exact input/output behavior has not been independently verified. ZAP alerts remain unvalidated, and no dedicated Broken Authentication test is claimed. The recommended next steps are to validate the outstanding observations, implement suitable controls in a controlled exercise, and retest using the same baseline and test cases.

> Overall limitation: this report documents educational lab results, not a production security audit. No remediation is represented as completed.

14. References

- OWASP Top 10 — https://owasp.org/www-project-top-ten/
- OWASP Web Security Testing Guide — https://owasp.org/www-project-web-security-testing-guide/
- OWASP SQL Injection Prevention Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP Cross Site Scripting Prevention Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP ZAP Documentation — https://www.zaproxy.org/docs/
- PortSwigger Web Security Academy — https://portswigger.net/web-security

---

Document: Week 6 — Web Security & Vulnerability Testing | DG Interns Hub Cybersecurity Internship | SAMYUKTHA S. | 13 October 2026